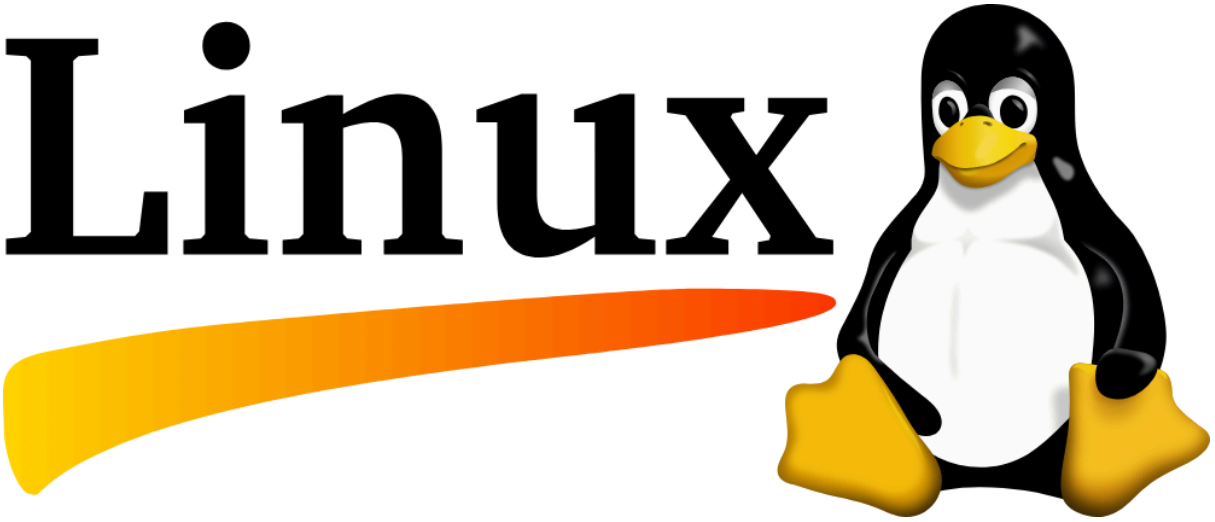
7/14 1회차 스터디



https://github.com/likelion-server-study/WEEK1

▼ 2주차 미리보기

다음 2주차는 PetClinic을 Docker 컨테이너로 실행해보는 방식으로 진행하려고 합니다.

단순히 이미지를 만들고 실행하는 것에서 끝내지 않고, 서버를 운영할 때 실제로 확인할 만한 내용까지 같이 다뤄보면 좋겠습니다.

2주차에 확인할 문제

- 다른 환경에서도 같은 서버를 실행할 수 있는가

  → PetClinic을 Docker 이미지로 만들고, 컨테이너로 실행해봅니다.

- 로컬에서 실행한 서버와 컨테이너 서버를 동시에 띄울 수 있는가

  → 컨테이너 내부 서버는 8080으로 두고, 내 컴퓨터에서는 8081로 접속하도록 연결합니다.

  → `8081:8080`은 "내 컴퓨터의 8081 요청을 컨테이너 내부 8080 서버로 전달한다"는 뜻입니다.

- 서버가 갑자기 죽으면 어떻게 되는가

  → 자동 재시작 설정 후 컨테이너를 강제 종료하고, 다시 살아나는지 확인합니다.

- 서버가 컴퓨터 자원을 너무 많이 쓰면 어떻게 제한하는가

→ CPU·메모리 제한을 걸고 `docker stats`로 사용량을 확인합니다.

- 서버가 열리지 않으면 어디를 확인해야 하는가

  → `docker ps`로 실행 상태, `docker logs`로 서버 로그, `docker inspect`로 포트·재시작·자원 설정을 확인합니다.

- 만든 이미지를 다른 사람 컴퓨터에서도 실행할 수 있는가

  → `docker save`로 이미지 파일을 만들고, 다른 컴퓨터에서 `docker load` 후 실행해봅니다.

  → 시간이 되면 Docker Hub에 올리고 내려받는 방식도 선택 실습으로 진행합니다.

▼ 간단한 진행 흐름 (예시)

1. Docker Desktop이 실행되어 있는지 확인

```
docker --version
docker info
```

2. Gradle로 PetClinic 이미지 만들기

```
.\gradlew.bat bootBuildImage --imageName=spring-petcli
nic:1.0
```

3. 재시작 정책과 CPU·메모리 제한을 걸고 실행

```
docker run -d --name petclinic-app --restart unless-st
opped --memory 1g --cpus 1 -p 8081:8080 spring-petclin
ic:1.0
```

4. 브라우저에서 서버 확인

```
<http://localhost:8081>
<http://localhost:8081/actuator/health>
```

5. 상태·로그·자원 사용량 확인

```
docker ps
docker logs --tail 80 petclinic-app
docker stats petclinic-app
docker inspect petclinic-app
```

6. 컨테이너를 강제 종료한 뒤 자동 재시작되는지 확인

```
docker kill petclinic-app
docker ps
```

7. 이미지 파일로 만들어 다른 컴퓨터에서 실행

```
docker save -o spring-petclinic-1.0.tar spring-petclin
ic:1.0
```

→ 다른 컴퓨터에서는 아래처럼 실행하면 됩니다.

```
docker load -i spring-petclinic-1.0.tar
docker run --name petclinic-app -p 8081:8080 spring-pe
tclinic:1.0
```

3주차 목표는 “Docker 명령어를 외우기”보다, 서버를 이미지로 묶은 뒤 실행 상태·로그·포트·자원·재시작을 어떻게 관리하는지 직접 확인하는 것입니다.